

第13章 类(对象)间关系

13.1 类(对象)间关系

在面向对象程序设计中，通常会经过分析、建模和设计等过程，抽象出多个类。在编码阶段，通过这些类及其实例化对象之间的交互，实现程序功能。从软件复用的角度来看，类(对象)之间的关系应尽可能松散，理想状态是彼此无关，这样更有助于类和对象的复用。然而，在实际开发中，完全无关的情况几乎不存在。类(对象)之间通常存在一定的关系，这种关系可以是单向的，也可以是双向的，其中，单向关系更有利于复用。讨论类(对象)间的关系时，涉及两个方面：物理上的关系和逻辑上的关系。

13.2 编译期依赖性

在 C++ 中，类的定义和实现通常分别编写在不同的文件中，类之间的物理关系则体现在这些文件之间的关联上。由于文件是编译器的输入单元，因此文件间的物理关系具体表现为编译期的依赖关系。

13.2.1 强关联

如果 A.h 必须包含 B.h 才能成功编译，那么 A 与 B 之间存在强关联。例如：

```
1 //A.h 文件
2 #include "B.h" //前置声明 class B;不行
3 class A {
4     private:
5         B    mB;
6     };

```

这里，类 A 中存在对象类型的成员 mB，需要包含相应的头文件才能通过编译。因此，建议尽量避免直接定义对象成员，可以使用指针成员代替，从而可以通过前置声明来降低编译期依赖性。又例如：

```
1 #include "B.h"
2 class A {
3     public:
4         void f( B * pB ) { pB->g( ); }
5     };

```

这里，在类 A 的成员函数 `f(B *pB)` 中，由于需要调用类 B 的成员函数 `g()`，因此必须包含类 B 的头文件以确保编译通过。由此可以看出，直接以内联方式实现成员函数可能会增加编译期的依赖性。为了避免这种问题，可以考虑使用前置声明并将函数定义改为外联实现。

13.2.2 硬关联

如果类 A 和类 B 之间存在双向关系，并且二者之间有强关联，那么可以认为 A 与 B 之间存在硬关联。显然，这种依赖性较强，在程序设计中应尽量避免此类情况的发生。例如：

```
1 // A.h
2 #include "B.h"
3 class A {
4 public:
5     A( );
6     A( int );
7 private:
8     B mB;
9 };
10 // B.h
11 class A;
12 class B {
13 public:
14     B( int );
15     A Foo( );
16 private:
17     int mData;
18 };
```

13.2.3 弱关联

如果 `A.cpp` 必须包含 `B.h` 才能编译通过，那么 A 与 B 之间存在弱关联。这种情况是最为常见的。例如：

```
1 // A.h
2 class B;
3 class A {
4 public:
5     A(int);
6     B func( );
7 private:
8     int mData;
9 };
```

```

10 // B.h
11 class B {
12 public:
13     B( );
14     B(int );
15 };
16 // A.cpp
17 #include "A.h"
18 #include "B.h"
19 B A::func( ) {
20     return B(mData);
21 }

```

13.2.4 软关联

在类 A 的定义和实现中，如果仅使用 B 的指针或引用，则 A 与 B 之间存在软关联。这种设计使得编译期依赖性降到最低。如果条件允许，应优先考虑采用这种方式。其主要特点包括：使用指针代替对象作为数据成员，函数参数通过指针或引用来传递，且函数返回对象时避免直接返回值对象。例如：

```

1 // A.h
2 class B;
3 class A {
4 public:
5     A( B * pb);
6     B & func01( );
7     B * func02( B & aB );
8 private:
9     B * mpB;
10 };

```

13.2.5 数据类型的选择

从硬关联、强关联、弱关联到软关联，文件的编译期依赖性依次减弱，其中，弱关联最为常见，而软关联则是最优选择。这几种关联形式在代码中的差异主要体现在数据类型的选择上，具体表现就是在数据成员、函数参数和返回值的类型设计上，我们可以选择对象、指针或引用等不同的形式。

虽然为降低编译期依赖性，可以优先选择指针或引用形式，但既然语言提供了对象、指针和引用等多种类型选择，每种形式自然各有优劣。接下来，我们将针对自定义类型的成员变量、函数参数和返回值进行比较说明。

1. 自定义类型的数据成员

类中的数据成员，可以是对象型、引用型或指针型，如：

```
1  class A {
2  private:
3      B    mB;    //对象型
4      B & mRefB; //引用型
5      B *  mpB;   //指针型
6  };
```

下表 13-1 给出不同形式的数据成员的对比说明。

表 13-1 不同类型数据成员的比较

成员类型	优点	缺点
对象型	不需要自定义类 A 的析构，拷贝构造，赋值函数； 自定义构造函数可选；	定义类 A 前，必须有类 B 的完整定义，即需要包含 B 的头文件，编译期依赖性强。
引用型	内部实现就是指针，占用空间少； 明确要求非空对象；	须使用初始化列表； 禁止赋值，编译器也不提供缺省赋值函数； 可能需要自定义拷贝构造函数； 对象存活期间 mRefB 必须存在且有效；
指针型	mpB 指向对象的生存期完全由程序员控制，更灵活；	可能需要自定义构造、拷贝、赋值、析构等函数；

这里特别说明一下引用型数据成员导致禁止赋值的问题。考虑类 A：

```
1  class A {
2      B & mRefB;
3  };
```

假设我们有两个 A 类对象 a1 和 a2，其中 a1 持有一个名为 mRefB 的 B 类对象的引用，而 a2 持有另一个名为 mRefB 的 B 类对象的引用。这两个引用可以指向不同的 B 类对象。

当执行 a1 = a2; 时，那么根据引用的含义，赋值应该是起外号的过程，也就是给 a2 中外号为 mRefB 的 B 类对象，在 a1 对象中另起一个外号 a1.mRefB，使得同一个 B 类对象具有两个外号，分别是 a1.mRefB 和 a2.mRefB。然而，C++ 中的引用具有一个重要的特性：一旦引用被初始化，它就始终绑定到同一个对象上，无法再重新绑定到其他对象。这意味着，在 a1 初始化后，它的成员 mRefB 已经绑定到了某个具体的 B 类对象，此后不能再改变这个绑定。所以，对于 a1 = a2;，如果能执行，也是将 a2 中 mRefB 对应的对象经过 B 类的赋值函数，赋值给 a1 中的 mRefB 对应的对象，实质就是两个 B 类对象之间的赋值，完全没有起外号的含义了。

因此，当类中包含引用类型的成员时，无论是编译器提供的缺省赋值函数，还是用户自定义的

赋值函数，都无法实现合理的赋值语义。为避免潜在的错误，C++语言明确规定，如果类中存在引用类型的成员，则该类不支持赋值操作，编译器不会为这样的类提供缺省的赋值函数，并且也不允许用户自定义赋值函数。

这也是在实际编程中，数据成员通常只使用对象类型或指针类型的原因之一。

2. 自定义类型的函数参数

在传递参数时，如果采用传值的方式传递对象，则会调用拷贝构造函数，这不仅效率较低，还要求类必须提供拷贝构造函数。相比之下，传引用和传指针的效率更高。其中，传指针允许实参为 `nullptr` 或有效对象的地址，适用于实参对象可有可无的情况；而传引用则要求实参必须是一个有效的对象。例如：

- `void study(Book &aBook)` 表示学习时必须使用一本书。
- `void study(Book *pBook)` 表示学习时可以选择使用一本书或不使用书。

可见，使用指针或引用型参数能够更清晰地表达参数的可选性和必要性。例如：

```

1  class A {
2  public:
3      void f1( B  aB );           //一定通过 B 类的拷贝构造函数，在栈区创建实参的副本对象 aB
4      void f2( B & aB );          //实参必须存在,实际传递的是一个指针
5      void f3( const B & aB );    //实参必须存在,实际传递的是一个指针
6      void f4( B * pB );          //实参可以是 nullptr，也可以非空
7      void f5( const B * pB );    //实参可以是 nullptr，也可以非空
8  };

```

3. 自定义类型的函数返回值

函数返回时，返回值的类型没有特殊限定，完全由函数的语义决定。但应该清楚：按值对象返回时，在效率上最低，而且隐含要求返回类型的拷贝构造函数必须可用；按引用返回时，返回的对象必须在函数返回后仍然存在且有效。

13.3 类(对象)间的逻辑关系

对于类(对象)间物理意义上的关系，在 C++语言中体现为编译期依赖性。这种依赖性与语言及编译器紧密相关的，不是所有语言都存在这种依赖性。通常所说的类(对象)间的关系，更多地是指类(对象)间的逻辑关系。这些逻辑关系与具体编程语言无关。逻辑关系是进行面向对象程序设计的依据，在软件分析、设计和编码的整个过程中都需要确定类(对象)间的逻辑关系，并且应保持前后一致。

类(对象)之间的逻辑关系可以从两个维度来理解：水平方向和垂直方向。这里我们先专注于水平方向上的基本逻辑关系，暂不涉及继承、模板等概念。在水平方向上，类之间的逻辑关系在 C++ 代码中主要表现为以下几种形式之一：

```
1 //形式 1：函数参数用到类 B
2 class A {
3     void f( B & aB );
4 };
5 //形式 2：函数返回值用到类 B
6 class A {
7     B * f( );
8 };
9 //形式 3：函数实现中用到类 B
10 class A {
11     int f( int n ) { B aB(n); return aB.func( ); }
12 };
13 //形式 4：数据成员用到类 B
14 class A {
15     B * mpB;
16 };
```

形式 1，2，3 的情况，称类 A 的实现依赖于 B 类的实现，简称依赖关系。形式 4 的情况，定义类 A 的数据成员时需要类 B，即类 A 需要关联类 B，简称关联关系。从关系的强弱程度看，依赖关系弱，关联关系强，因此依赖关系也是最常见，使用最多的逻辑关系。

13.4 实例间的依赖关系

依赖关系体现在实现某个成员函数时需要使用另一个类的对象，表明一个对象的行为实现依赖于另一个对象，反映了对象间的行为联系，属于实例级别的关联。依赖关系既可以是单向的，也可以是双向的。依赖关系的 UML 图例如图 13-1。

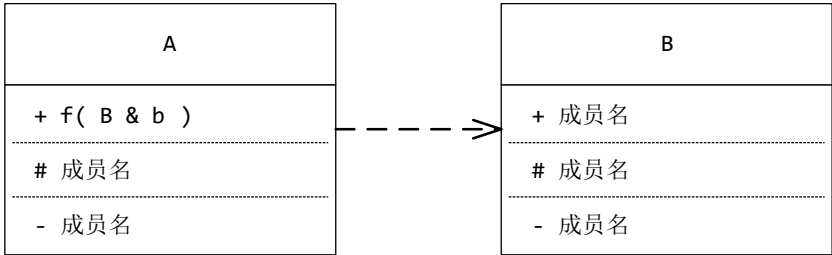


图 13-1 依赖关系的 UML 图例

下面给出一个示例：老鼠吃果园中的苹果，每吃一个苹果，苹果所含能量按比例转变成老鼠的

新增体重。

老鼠和苹果之间通过"吃"的行为联系起来,要完成"吃"的行为,需要两个事物参与;增加体重是"吃"的行为后果;"吃"的行为可以放在老鼠类,也可以放在苹果类,甚至两个类中都放;一般把行为放在主动多变的类中。这里"吃"的主动方是老鼠,将"吃"的行为定义在老鼠类更符合习惯,但这不是绝对的。再如有多个品种的老鼠,每种老鼠"吃"苹果的方式不一样:大老鼠吃整个苹果,小老鼠只吃苹果肉,那么老鼠就是多变的,将"吃"的行为定义在老鼠类更好一些。又如多个品种的老鼠,每种老鼠"吃"苹果的算法不一样:大老鼠新增体重缓慢,小老鼠新增体重快速,那么老鼠还是属于多变的一方,也是最好将"吃"的行为定义在老鼠类。另一方面,如果有多个品种的苹果,各种苹果被老鼠吃的后果不一样:高档苹果所含能量高,普通苹果能量低,那么苹果就成为多变的一方,这时将"被吃"定义在苹果类中更合适。

例如:在 Mouse 中定义 eat, 比例系数 Apple::factor()由 Mouse 和 Apple 共同决定。

```

1  class Apple;
2  class Mouse {
3  public:
4      Mouse( int w):mWight(w) {}
5      void eat(Apple & apple);
6      int getWeight( ) const { return mWeight;}
7  private:
8      int  mWeight = 100 ;
9  };
10 class Apple {
11 public:
12     Apple(int e):mEnergy(e) {}
13     int getEnergy( ) const { return mEnergy; }
14     float factor(Mouse & m); //由 Mouse 和 Apple 共同决定比例系数
15 private:
16     int  mEnergy;
17 };
18 void Mouse::eat(Apple & apple) {
19     mWeight += apple.getEnergy( ) * apple.factor(*this);
20 }
21 float Apple::factor(Mouse & m) {
22     if ( m.getWeight( ) >= 100 )
23         return mEnergy>10 ? 0.001 : 0.015;
24     else
25         return mEnergy>20 ? 0.002 : 0.025;
26 }
```

又例如同时定义 eat 和 eaten 函数，可修改 Apple 类定义：

```

1  ...
2  class Apple {
3  public:
4      Apple(int e):mEnergy(e) {}
5      int getEnergy( ) const { return mEnergy; }
6      float factor( Mouse & m );    //由 Mouse 和 Apple 共同决定比例系数
7      void eaten( Mouse & m ) {
8          m.eat( *this );
9      }
10 private:
11     int mEnergy;
12 };

```

13.5 类间关联关系

依赖关系表现为只在"需要"对象的时候，才产生"依赖"。但当类中含有类的成员的时候，例如：

```

1  class A {
2      B * mpB;
3  };

```

这时强调的是一种"非偶然性"的知道，即类 A 的对象只要存活，就需要"知道"一个类 B 对象，哪怕是一个空的类 B 对象，也算知道。称这样的类 A 和类 B 之间，具有关联关系(Association)。关联关系的 UML 图例如图 13-2。

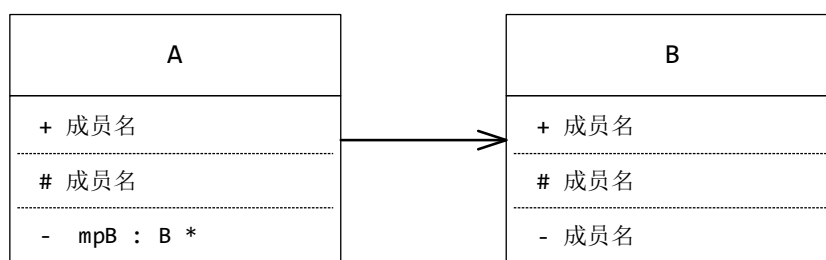


图 13-2 关联关系的 UML 图例

关联关系又可分为普通关联关系和聚集关系。普通关联关系表示两个类之间，关系"平等"，没有包含的逻辑含义。一般所说的关联关系就是这种关系。另一种关联关系称为聚集关系，虽然也是关联，但更强调两个类之间具有整体和部分的关系，两个类之间的地位是"不平等"的。聚集关系进一步又分为组合关系和聚合关系。

普通关联是常见的类间关系，既可以是单向的，也可以是双向的；既可以是一对一的，也可以

是一对多或多对一的，甚至多对多的。

例如，强调一个 Student 总是"知道"其所在学校以及各门课程：

```
1 class Student {
2     private:
3         University * mpUniversity;
4         Course      * mpCourses[30];
5     };
```

再例如，下面自关联的 Student 类强调一个 Student 总是"知道"其同寝室的同学以及班级中的学霸：

```
1 class class Student {
2     private:
3         Student * mpMates[3];
4         Student * mpSuper;
5     };
```

关联类也是关联关系的一种常见表现形式。两个类之间的联系通常较为直观且简单，如 A 知道 B，A 属于 B，A 拥有 B，A 用到 B 等，但也存在更复杂的联系，例如企业与雇员之间的雇佣和被雇佣关系，雇佣关系本身可包括各种数据和行为，如薪资、职位、岗位职责的数据，以及调薪、调职等行为，这时可将雇佣关系单独抽象为一个或多个关联类。再比如，教师和学生之间通过课程 (Course) 联系在一起的，就可以将这种关联联系抽象为一个关联类。如：

```
1 class Course {
2     //课程名，开课学期，授课地址，学分，教材等信息
3 };
4 class Teacher {
5     private:
6         Course * mpCourse;
7 };
8 class Student {
9     private:
10        Course * mpCourse;
11 };
```

13.6 聚集关系

在关联关系中，如果要强调二者是整体和部分之间的逻辑关系，那么此时的关联关系称为聚集关系，聚集关系中又根据整体对部分的控制能力，分为组合和聚合。

13.6.1 组合(Composition)

假设类 A 和类 B 之间是关联关系，且类 A 和类 B 之间是整体与部分的关系，同时整体负责各部分的创建和消亡，那么此时类 A 和类 B 之间的关系，称为组合。组合关系的 UML 图例如图 13-3。

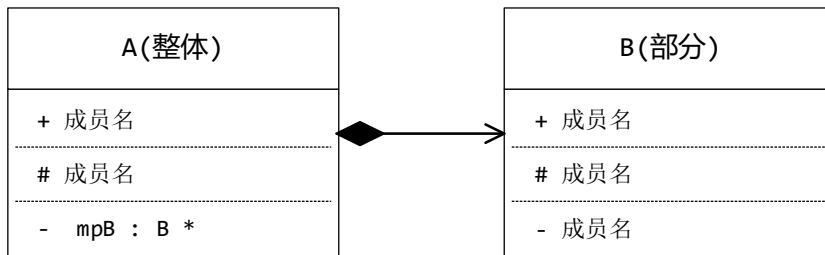


图 13-3 组合关系的 UML 图例

例如：

```

1  class Person {
2  private:
3      Head  mHead;
4  };
  
```

这里将 Person 看作整体，Head 看作部分；创建一个 Person 时，必将创建 Head，释放 Person 时，必将释放 Head；创建和释放 Head 的时机完全由 Person 控制，所以可以说 Person 组合 Head。要在代码的实现上体现这种组合关系，除了示例中那样实现，还可以使用其它的实现方式，如：

```

1  class Person {
2  public:
3      Person( ) : mpHead( new Head ) {}
4      ~Person( ) { delete mpHead; }
5      Person( const Person & ) { /* 略 */}
6      Person & operator=( const Person & ) { /* 略 */}
7  private:
8      Head * mpHead;
9  };
10 或
11 class Person {
12 public:
13     Person( ) : mpHead( std::make_shared<Head>( ) ) { }
14 private:
15     std::shared_ptr<Head> mpHead;
16 };
  
```

在组合关系中，要求整体负责各部分的创建和消亡，上边例子中 Person 和 Head 属于整体和部

分"同生共死",但组合并不要求一定"同生共死",只要求完全控制部分的生死。例如,下面也属于组合关系:

```
1 class Person {
2 public:
3     Person( ):mpHead(new Head) {}
4     ~Person( ) { delete mpHead; }
5     Person( const Person & ) { /* 略 */}
6     Person & operator=( const Person&) { /* 略 */}
7     void changeHead( ) {
8         delete mpHead;
9         mpHead = new Head;
10    }
11 private:
12     Head * mpHead;
13 };
```

在 Person 的存活期间,可以释放已有的 Head,并创建新的 Head。Head 对象与 Person 对象并不一定是同生共死的,但 Head 对象的生命周期总是由 Person 控制的,这也属于组合关系。

13.6.2 聚合(Aggregation)

假设类 A 和类 B 之间是关联关系,同时强调类 A 和类 B 之间是整体与部分的关系,但整体不负责各部分的创建和消亡,那么此时类 A 和类 B 之间的关系,称为聚合。此时整体和各部分的生存期是独立的,也就是整体存在时,部分可以不存在;部分存在时,整体也可以不存在。聚合关系的 UML 图例如图 13-4。

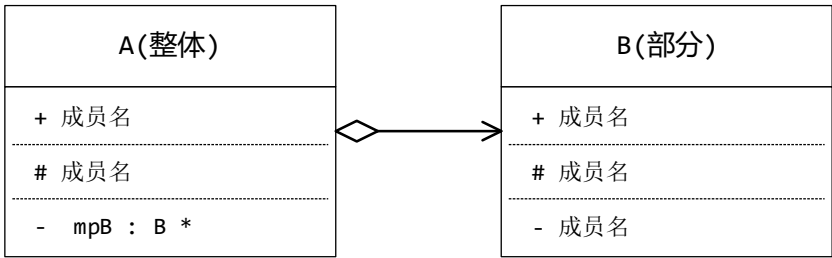


图 13-4 聚合关系的 UML 图例

例如,把人(Person)看作整体,把宠物(Pet)看作部分。当 Person 存在时, Pet 可有可无; Pet 存在时, Person 也可有可无,那么 Person 类和 Pet 类之间就是聚合关系,代码表示为:

```
1 class Person {
2 public:
```

```

3     Person( Pet & aPet) : mpPet(&aPet) { }
4     ~Person( ) = default;
5     Person( const Person & ) { /* 略 */ }
6     Person & operator=( const Person&) { /* 略 */ }
7 private:
8     // Person 聚合 Pet。虽然构造 Person 时, Pet 须存在, 但 Person 不负责 Pet 释放
9     Pet * mpPet;
10 };
11 或者
12 class Person {
13 public:
14     Person( ) : mpPet( nullptr) { }
15     ~Person( ) = default;
16     Person( const Person & ) { /* 略 */ }
17     Person & operator=( const Person&) { /* 略 */ }
18 public:
19     void setPet( Pet * pPet) { mpPet = pPet; }
20 private:
21     // Person 聚合 Pet。
22     // Person 存活期, 可通过 setPet 指定或清空 Pet, 但不需负责 Pet 的创建和释放
23     Pet * mpPet;
24 };

```

13.6.3 组合/聚合的选择

组合和聚合只适用于强调整体和部分关系的时候, 在具体选择使用组合还是聚合时, 应基于以下几点考虑:

1. 选择在问题域中更适合的。

例如, 汽车和车轮的关系, 一般常识认为汽车总是有车轮的, 因此, 采用汽车组合车轮的关系更为普遍。但如果开发汽车维修管理程序, 那么在维修过程中, 虽然车轮仍是汽车的一部分, 但车轮对象通常需要独立于汽车对象存在。这时使用聚合关系表示汽车和车轮之间的逻辑关系会更合适。

2. 选择更符合认知常识的。

例如, Person 和 Head 的关系, 常识认为一个 Person 总是有 Head 的, 存在一个 Head 总是应该对应地存在一个 Person, 因此采用 Person 组合 Head 更适合。下面的代码可作为反例:

```

1 class Person {
2 public:
3     Person(Head * pHead):mpHead(pHead) { }
4 private:

```

```

5     Head * mpHead;
6 };
7 int main( ) {
8     Head head;
9     Person person(&head);
10 }

```

程序中 Person 类聚合 Head 类，并且程序也能正常运行。但此时的 Person 类隐含地允许存在没有 Head 的 Person 对象，或者存在不属于任何 Person 的 Head 对象，这显然是不合理的。你可能会认为，在 main()方法中总是先创建 Head 对象再将其赋给 Person 对象，从而确保每个 Person 都有一个 Head。然而，这种保证依赖于特定的上下文环境。在进行类设计时，考虑到复用性，不应对类的应用场景和使用环境做过多的限制。因此这里使用聚合关系是不恰当的，即使程序能够正常运行，这样的设计也应尽量避免。

3. 考虑代码实现上的难易程度。

仍考虑 Person 和 Head 的组合关系，例如代码：

```

1 class Person {
2 private:
3     Head mHead;
4 };

```

如此定义 Person 类，会隐含地要求 Head 类有构造、拷贝、赋值、析构等函数。如果，Head 类的构造、拷贝和赋值是极其消耗系统资源的，那会导致 Person 类在使用时效率低下；再比如，如果 Head 类禁止了拷贝构造和赋值，那么缺省情况下，Person 类的拷贝和赋值也会被禁止，要允许拷贝和赋值，必须自定义 Person 类的拷贝和赋值，但又如何实现 Person 类中 Head 的拷贝和赋值语义呢？如果在代码实现上存在类似的限制条件，那么仍使用组合关系可能就不合适了，改为其它关系会更合适，如普通关联关系。

4. 考虑代码实现对语义的隐含要求。

例如，Person 和 Pet 的聚合关系，代码如下：

```

1 class Person {
2 public:
3     Person( Pet & aPet) : mpPet(&aPet) { }
4     ~Person( ) = default;
5     Person( const Person & ) { /* 略 */}
6     Person & operator=( const Person&) { /* 略 */}
7 private:
8     Pet * mpPet;

```

```
9    };
```

这里, `Person` 聚合 `Pet`, `Person` 创建前, `Pet` 一定存在, `Person` 释放后, `Pet` 仍存在。但这里的 `Person` 类还有一个隐含的语义要求: `Person` 存活时, `Pet` 一定活着! 也就是说, 这样定义的 `Person` 类, 隐含要求 `mpPet` 非空且有效。如果不能保证这点, 那么采用聚合关系就是不恰当的或着说设计是存在缺陷的, 此时, 可考虑修改关系为普通关联或组合。

13.7 例 1— 工作簿的组成

现定义简化的 Excel 工作簿。一个工作簿(`Book`)由多个工作表单(`Sheet`)组成, 表单为多个单元格(`Cell`)组成的矩形, 一行(`Row`)中的所有单元格组成一行, 一列(`Column`)中的所有单元格组成一列, 同时, 表单中还可以含有多个区域(`Range`), 每个区域是由多个单元格组成的矩形。

从需求中易得主要的类: `Book`, `Sheet`, `Cell`, `Row`, `Column` 和 `Range`。分析它们之间的关系: `Book` 适合组合多个 `Sheet`; `Sheet` 组合多个 `Row`, `Sheet` 组合多个 `Column`, `Sheet` 组合多个 `Range`。

关键是如何定义 `Sheet` 与 `Cell` 之间的关系, 以及 `Row`、`Column`、`Range` 与 `Cell` 之间的关系。如果 `Sheet` 组合 `Cell`, 那么 `Cell` 应由 `Sheet` 负责创建和释放, 此时 `Row`、`Column`、`Range` 就不能再负责创建和释放 `Cell` 了, 因此, `Row`、`Column`、`Range` 应聚合 `Cell`。类间关系如图 13-5 所示:

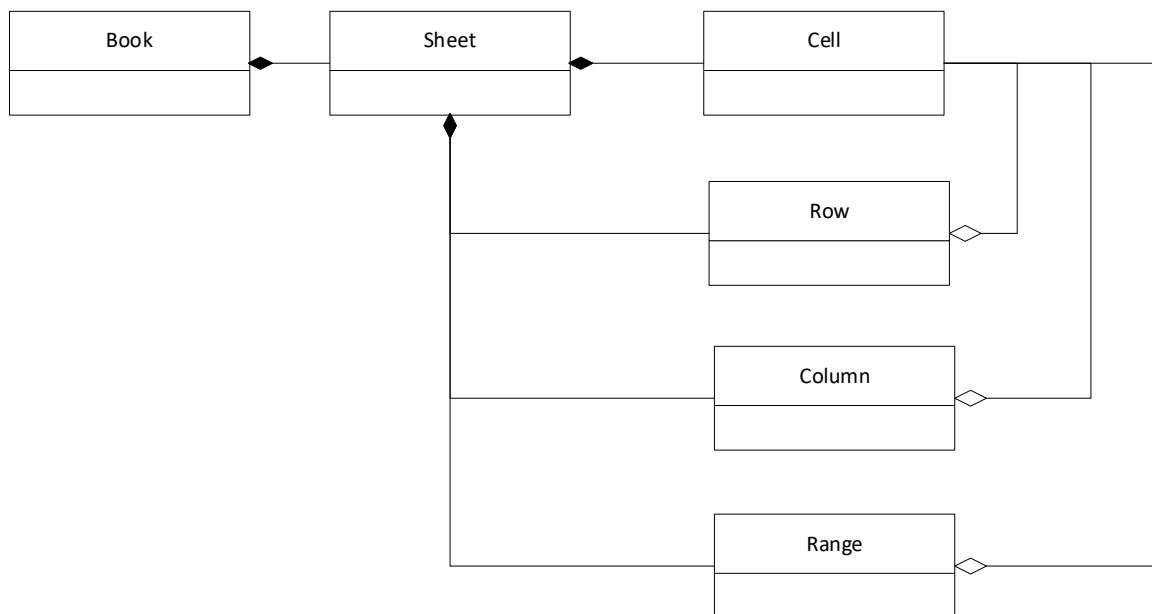


图 13-5 `Book`、`Sheet`、`Cell` 等类的 UML 类图

如果 `Sheet` 聚合了 `Cell`, 那么 `Sheet` 将不再负责创建和释放 `Cell`, 此时必须有一个类来承担这一职责。然而, 由于一个 `Cell` 可能属于不同的 `Row`、`Column` 或 `Range`, 也可能仅属于某一行而不在任何 `Range` 中, 因此将创建和释放 `Cell` 的责任交给 `Row`、`Column` 或 `Range` 中的任何一个类都不合适。

若让这三个类共同负责,则可能导致重复释放的问题。为了解决这一困境,可以改进设计,例如引入一个专门负责创建和释放 Cell 的新类——CellMgr 类。

总体而言,由于采用 Sheet 聚合 Cell 的设计方式通常比 Sheet 组合 Cell 更为复杂,因此在本例中,Sheet 组合 Cell 的设计方案将是优先选择。

剩下可能令人困扰的是:如何从一个 Cell 获知其所在的 Sheet、Row、Column 以及 Range? 最直接的方式是让 Cell 持有对这些对象的普通关联,即在构建 Cell 时或通过 setter 方法进行指定。例如:

```

1  class Cell {
2  public:
3      void setSheet(Sheet * p) { mpSheet = p;}
4      void setRow(Row * p)      { mpRow = p;}
5      void setColumn(Column * p){ mpColumn = p;}
6      void setRange(Range * p) { mpRange = p;}
7      //其它略
8  private:
9      Sheet * mpSheet;
10     Row   * mpRow;
11     Column* mpColumn;
12     Range * mpRange;
13 };

```

另一种方式是通过 Sheet 对象间接获取当前单元格所在的行、列及区域等信息。例如:

```

1  class Cell {
2  public:
3      void setSheet(Sheet * p) { mpSheet = p;}
4      Row * getRow( ) const    { return mpSheet->getRow(x,y); }
5      Column * getColumn( ) const { return mpSheet->getColumn(x,y); }
6      Range * getRange( ) const { return mpSheet->getRange(x,y); }
7      //其它略
8  private:
9      Sheet * mpSheet;
10     int x;
11     int y;
12 };

```

甚至可以进一步解除对 Sheet 的直接关联,转而通过单例 SheetMgr 获取当前 Sheet,从而将原本的关联关系替换为更弱的依赖关系。例如:

```

1  class Cell {

```

```

2 public:
3     Sheet * getSheet( ) const    { return SheetMgr::getInstance( ).activeSheet( ); }
4     Row    * getRow( ) const     { return getSheet( )->getRow(x,y); }
5     Column * getColumn( ) const  { return getSheet( )->getColumn(x,y); }
6     Range  * getRange( ) const   { return getSheet( )->getRange(x,y); }
7     //其它略
8 }

```

从本例也可以看出,在进行面向对象设计时,确定合适的类间关系至关重要。合理的关系设计不仅有助于提升系统的可理解性,也为后续的代码实现打下良好基础。相比具体的代码实现,类间关系的设计更为关键且具有指导意义,因为实现细节往往因技术环境或需求变化而不同,存在多种可能性。因此,在设计阶段应优先关注类之间的关系建模,而非具体功能的编码实现。

13.8 例 2— 警察和警察局

现模拟实现一个虚拟场景:警察局有多名警察,警察可以抓捕窃贼。每当警察抓获一名窃贼,不仅能提升个人年终奖金,还会提高所属警察局的声望。

根据这一描述,我们可以抽象出三个类:警察局(PoliceStation)、警察(Police)和窃贼(Thief)。警察与窃贼之间存在依赖关系,但警察局与警察之间的关系更为复杂。

警察局与警察之间的关系分为两个方向,一个是从警察到警察局,另一个是从警察局到警察。无论哪个方向,都不适合用依赖关系来表示,而应采用关联、组合或聚合关系。

如果只考虑单向关系,理论上我们有 $3 + 3 = 6$ 种选择;如果是双向关系,则有 $3 \times 3 = 9$ 种可能的组合方案。具体选择哪种关系,需要根据实际应用场景的需求来决定。以下是针对不同需求场景的几种示例:

场景 1: 警察局知道其有哪些警察,每个警察也需要知道其所在的警察局。若把警察局和警察视为整体-部分关系,那么,警察局到警察可以是聚合或者组合关系,而警察和警察局就不能是整体-部分关系了,只能是普通关联。具体地:

1) 警察局组合警察

警察局和警察视为整体-部分关系,警察局负责各警察的创建和销毁,这隐含地表明不存在不属于任何警察局的警察。如图 13-6 所示:



图 13-6 警察局组合警察的 UML 类图

具体的实现代码可如下：

```

1 // Thief.h
2 #pragma once
3 class Thief
4 { };
1 // Police.h
2 #pragma once
3 #include <string>
4 class PoliceStation;
5 class Thief;
6 class Police {
7     friend class PoliceStation;
8 private:
9     //私有的构造函数,确保每个警察只能由非空的警察局创建和维护
10    Police( const std::string & name, PoliceStation & ps )
11        : mName( name ), mpStation( &ps ), mPrize( 0 ) { }
12    Police( const Police & ) = delete; //禁止拷贝
13    Police & operator=( const Police & ) = delete; //禁止赋值
14 public:
15    void capture( Thief & aThief );
16    const std::string & name( ) const { return mName; }
17    int prize( ) const { return mPrize; }
18 private:
19    const std::string&mName;
20    PoliceStation *      mpStation; //必定非空, 且不能以对象形式定义
21    int                  mPrize = 0;
22 };
  
```

```

1 // Police.cpp
2 #include "Police.h"
3 #include "PoliceStation.h"
4 void Police::capture( Thief & aThief ) {
5     if( mpStation ) mpStation->addReputation( 10 ); //增加警察局声望
6     mPrize += 200; //增加个人奖金
  
```

```
7 }
```

```
1 // PoliceStation.h
2 #pragma once
3 #include <string>
4 #include<unordered_map>
5 class Police;
6 class PoliceStation {
7 public:
8     PoliceStation( ) = default;
9     PoliceStation( const PoliceStation & ) = delete;           //禁止拷贝
10    PoliceStation & operator=( const PoliceStation & ) = delete; //禁止赋值
11    ~PoliceStation( ); //外联实现
12 public:
13    void addReputation( int added ) { mReputation += added; }
14    void addPolice( const std::string & name ); //外联实现
15    void removePolice( const std::string & name ); //外联实现
16 public:
17    int reputation( ) const { return mReputation; }
18    const std::unordered_map<std::string, Police *> & polices( ) const
19        { return mPolices; }
20    Police * police( const std::string & name ) const; //外联实现
21 private:
22    int mReputation = 100; //声望
23    std::unordered_map<std::string, Police *> mPolices {}; //警察集合
24 };
```

```
1 // PoliceStation.cpp
2 #include "PoliceStation.h"
3 #include "Police.h"
4 PoliceStation::~~PoliceStation( ) {
5     for (auto pair : mPolices) delete pair.second;
6 }
7 Police * PoliceStation::police( const std::string & name ) const {
8     auto it = mPolices.find( name );
9     return (it != mPolices.end( )) ? it->second : nullptr;
10 }
11 void PoliceStation::addPolice( const std::string & name ) {
12     mPolices[ name ] = new Police( name, *this );
13 }
14 void PoliceStation::removePolice( const std::string & name ) {
15     auto it = mPolices.find( name );
```

```

16     if (it != mPolices.end( )) {
17         delete it->second;    //须释放对应的 Police
18         mPolices.erase( it );
19     }
20 }

```

```

1  // 测试用主函数
2  #include <iostream>
3  #include "Thief.h"
4  #include "Police.h"
5  #include "PoliceStation.h"
6  int main( )
7  {
8      PoliceStation s1;
9      PoliceStation s2;
10     s1.addPolice( "张三" );
11     s1.addPolice( "李四" );
12     s2.addPolice( "王五" );
13     //Police p4( "赵六" ); //不能存在不属于任何警察局的赵六
14     Thief t1;
15     Thief t2;
16     Thief t3;
17     Thief t4;
18     if (Police * p = s1.police( "张三" )) p->capture( t1 );
19     if (Police * p = s1.police( "张三" )) p->capture( t2 );
20     if (Police * p = s1.police( "李四" )) p->capture( t3 );
21     if (Police * p = s2.police( "王五" )) p->capture( t4 );
22     std::cout << "S1 Reputation = " << s1.reputation( ) << std::endl; //输出 130
23     std::cout << "S2 Reputation = " << s2.reputation( ) << std::endl; //输出 110
24     std::cout << "Polices in S1:" << std::endl;
25     for (auto pair : s1.polices( )) {
26         std::cout << pair.first << " = " << pair.second->prize( ) << std::endl;
27     }
28     std::cout << "Polices in S2:" << std::endl;
29     for (auto pair : s2.polices( )) {
30         std::cout << pair.first << " = " << pair.second->prize( ) << std::endl;
31     }
32 }

```

运行结果为:

```

1  S1 Reputation = 130
2  S2 Reputation = 110
3  Police in S1 :

```

```

4   张三 = 400
5   李四 = 200
6   Police in S2 :
7   王五 = 200

```

这里需要特别强调：对于某种逻辑关系，在代码实现上不是唯一的。上述代码只是体现组合关系的一种实现，其它的实现方式也是可以的。例如，如果将 `Police` 的构造函数改为 `public` 的，去掉友元类的声明，对于上边的主函数来说，仍然体现警察局和警察的组合关系。一个良好的设计应该既体现逻辑关系，又能降低用户在使用中意外违反逻辑关系的可能性。这里将 `Police` 的构造函数设为私有，就是为了避免用户端(如 `main` 函数)以错误的方式实例化警察对象，从而违背原设计中要体现的组合关系。

2) 警察局聚合警察

相较于组合，也可以让警察局聚合警察，此时仍将警察局和警察视为整体-部分关系，但警察局不需要负责各警察的创建和销毁，也就是警察局和警察可以有不同的生存期，甚至允许存在不属于任何警察局的警察。如图 13-7 所示：



图 13-7 警察局聚合警察的 UML 类图

在代码实现时，也需要做一些相应的修改，如：

```

1   // Police.h
2   #pragma once
3   #include <string>
4   class PoliceStation;
5   class Thief;
6   class Police {
7   public:
8       //所属的 PoliceStation 可以为空
9       Police( const std::string & name, PoliceStation * ps = nullptr)
10          : mName( name ), mpStation( ps ), mPrize( 0 ) { }
11       //其它同组合关系，略
12   };

```

```

1 // PoliceStation.h
2 #pragma once
3 #include <string>
4 #include <unordered_map>
5 class Police;
6 class PoliceStation {
7 public:
8     //其它同组合关系, 略
9     ~PoliceStation( ) = default;
10 public:
11     void addPolice( Police * p );           //外联实现
12     void removePolice( const std::string & name ); //外联实现
13     //其它同组合关系, 略
14 };

```

```

1 // PoliceStation.cpp
2 #include "PoliceStation.h"
3 #include "Police.h"
4 void PoliceStation::addPolice( Police * p ) {
5     if (p) mPolices[ p->name( ) ] = p;
6 }
7 void PoliceStation::removePolice( const std::string & name ) {
8     auto it = mPolices.find( name );
9     if (it != mPolices.end( )) mPolices.erase( it );
10 }
11 //其它同组合关系, 略

```

适用聚合关系的主函数:

```

1 #include <iostream>
2 #include "Thief.h"
3 #include "Police.h"
4 #include "PoliceStation.h"
5 int main( )
6 {
7     PoliceStation s1;
8     PoliceStation s2;
9     Police p1( "张三", &s1 );
10    Police p2( "李四", &s1 );
11    Police p3( "王五", &s2 );
12    Police p4( "赵六" );           //不属于任何警察局的赵六
13    s1.addPolice( &p1 );
14    s1.addPolice( &p2 );
15    s2.addPolice( &p3 );

```

```
16
17     //其它同组合关系，略
18
19     //输出结果与组合关系一样
20 }
```

在使用聚合关系时，应特别注意整体与部分之间关于部分对象的释放问题，例如此处各个警察对象的释放。在组合关系中，警察局负责释放其所包含的警察对象；但在聚合关系中，警察局不再承担这一职责。然而，警察对象的释放始终需要被妥善处理，通常有两种方式：一种是由使用方负责，例如将各警察作为局部变量交由 `main` 函数自动释放；另一种则是单独抽象出一个专门管理警察对象的类，例如 `PoliceMgr` 类，由该类在其生命周期结束时负责释放所有警察对象。

场景 2：不把警察局和警察视为整体-部分关系，也不把警察和警察局视为整体-部分关系。警察局知道其有哪些警察，每个警察需要知道其所在的警察局，也可以不属于任何警察局。这时，警察局和警察之间是一种双向关系，其中，警察到警察局的方向上是普通关联，警察局到警察的方向上也是普通关联。

此时的实现代码与警察局聚合警察的代码可完全一致，这也说明逻辑关系是存在于程序员头脑中的，确定逻辑关系后可编写出程序代码，但不一定能够从程序代码逆推出逻辑关系。

场景 3：将警察视为整体，将警察局视为部分。这时，警察组合警察局就不合适了，那将隐含地表示每个警察都拥有自己的警察局。可用警察聚合警察局，警察局关联警察，这时的实现代码也可以与警察局聚合警察的代码一样。

这里要指出：现实世界中，将警察视为整体而将警察局视为部分不符合人的常识性认知，但不代表程序设计中不能这么做。虽然在程序设计中推荐将现实世界的关系直接映射到设计中，方便阅读和理解，但这不是必须的，而是应根据问题域的具体需求确定类间的逻辑关系。例如，在代码中将头(`Head`)看作整体，而将人(`Person`)看作 `Head` 中信息的一部分，即 `Head` 组合 `Person`，也是可以的。